
```
>>> rects = [[10, 2], [3, 6], [2, 4], [3, 9], [10, 7], [9, 9]]
>>> sorted(rects, key=lambda rect: (rect[0] * 2) + (rect[1] * 2))
[[2, 4], [3, 6], [10, 2], [3, 9], [10, 7], [9, 9]]
```

Rather than sorting the values `[10, 2]` or `[3, 6]`, for example, the function now sorts based on the returned perimeter integers 24 and 18. Lambda functions are a convenient syntactic shortcut: you can specify a small one-line lambda function instead of defining a new, named function with a `def` statement.

Mapping and Filtering with List Comprehensions

In earlier Python versions, the `map()` and `filter()` functions were common higher-order functions that could transform and filter lists, often with the help of lambda functions. Mapping could create a list of values based on the values of another list. Filtering could create a list that contained only the values from another list that match some criteria.

For example, if you wanted to create a new list that had strings instead of the integers `[8, 16, 18, 19, 12, 1, 6, 7]`, you could pass that list and `lambda n: str(n)` to the `map()` function:

```
>>> mapObj = map(lambda n: str(n), [8, 16, 18, 19, 12, 1, 6, 7])
>>> list(mapObj)
['8', '16', '18', '19', '12', '1', '6', '7']
```

The `map()` function returns a `map` object, which we can get in list form by passing it to the `list()` function. The mapped list now contains string values based on the original list's integer values. The `filter()` function is similar, but here, the lambda function argument determines which items in the list remain (if the lambda function returns `True`) or are filtered out (if it returns `False`). For example, we could pass `lambda n: n % 2 == 0` to filter out any odd integers:

```
>>> filterObj = filter(lambda n: n % 2 == 0, [8, 16, 18, 19, 12, 1, 6, 7])
>>> list(filterObj)
[8, 16, 18, 12, 6]
```
